# AMANE — Refonte V3

Faire d'AMANE une enveloppe d'exécution souveraine pour SaaS externes

Journal de refonte, décisions d'architecture et tri du dépôt

| | |
|---|---|
| **Version** | 2.0 |
| **Date** | 2026-10-29 |
| **Branche** | main3 |
| **Statut** | Étape 4 terminée — étape 5 à suivre |

# Sommaire

**Fiche du document** 4

Historique des versions . . . 4

**1. Objet de la refonte** 5

1.1 Ce qu'est une « enveloppe d'exécution » . . . 5

1.2 La promesse à tenir . . . 5

**2. État actuel du dépôt (V2, point de départ)** 6

2.1 Point de départ figé . . . 6

2.2 Cartographie des composants . . . 6

2.3 Trois constats qui orientent le tri . . . 6

**3. Décisions d'architecture** 8

ADR-001 — Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go . . . 8

ADR-002 — Formats d'exécutables acceptés . . . 9

ADR-003 — Zero-knowledge strict . . . 9

ADR-004 — Modalités de la haute disponibilité PME . . . 9

ADR-005 — Installation de la pile HA chez la PME . . . 10

ADR-006 — Réglages de bascule . . . 11

ADR-007 — Hébergement de l'application externe . . . 12

ADR-008 — Confinement et chiffrement . . . 13

**4. Tri du dépôt** 15

4.1 À garder intact . . . 15

4.2 À garder mais adapter . . . 15

4.3 À supprimer . . . 16

**5. Plan de travail par étapes** 17

**6. Journal des étapes** 18

Étape 0 — Préparation (2026-10-28) . . . 18

Étape 1 — Nettoyage (2026-10-28) . . . 18

Étape 1 — Compléments (2026-10-28) . . . 19

Étape 2a — Image de l'orchestrateur et laboratoire HA (2026-10-28) . . . 19

Étape 2b — Pile PME réelle (2026-10-29) . . . 23

Méthode de travail à partir du 2026-10-29 . . . 26

Étape 2c — Recentrage du nœud Rust (2026-10-29) . . . 26

Étape 2d — Réplication synchrone, alerte et mesures (2026-10-29) . . . 27

Étape 3 — Le nœud, hôte d'exécution (2026-10-29) . . . 28

Étape 4 — Confinement et chiffrement (2026-10-29) . . . . . 29

**7. Points ouverts** **31**

**Annexe — Mettre à jour ce document** **33**

**Annexe — Validation sur le banc AMANE Test (étape 2b)** **34**

# Fiche du document

Ce document est le **journal de bord de la refonte V3** d'AMANE. Il récapitule l'état du dépôt au départ, le tri de chaque composant (garder intact, adapter, supprimer), les décisions d'architecture prises et leur justification, puis l'avancement étape par étape. Il sert de référence aux collaborateurs pour comprendre **ce qui a été fait, pourquoi, et dans quel ordre**.

Il est mis à jour à chaque étape de la refonte. La source fait foi (`docs/refonte-v3/AMANE-Refonte-V3.md`) ; le PDF est régénéré à partir d'elle (voir l'annexe « Mettre à jour ce document »).

## Historique des versions

| Version | Date | Branche / commit | Changements |
|---|---|---|---|
| 1.0 | 2026-10-28 | main3 / 6da5022 | Création : état V2, tri initial, décisions ADR-001 à ADR-003, plan par étapes |
| 1.1 | 2026-10-28 | main3 / dc06197 | Étape 1 (nettoyage) terminée ; tri corrigé (§ 4.3) ; points à vérifier ajoutés (§ 7) |
| 1.2 | 2026-10-28 | main3 | Compléments à l'étape 1 : compilation Rust confirmée, test de réplication corrigé, test V1 `mock_server.rs` retiré ; réplication synchrone ajoutée à ADR-001 et à l'étape 2 |
| 1.3 | 2026-10-28 | main3 | ADR-004 (modalités de la HA PME, 7 décisions) ; `quorum.rs` réorienté plutôt que supprimé ; étape 2 découpée en 2a à 2d |
| 1.4 | 2026-10-28 | main3 | Sous-étape 2a : image de l'orchestrateur, laboratoire HA trois machines, 10 scénarios de chaos réussis, 3 défauts de l'orchestrateur corrigés |
| 1.5 | 2026-10-28 | main3 | ADR-005 (installation de la pile HA chez la PME, 5 décisions) ; défaut zero-knowledge des mots de passe dérivés du jeton |
| 1.6 | 2026-10-29 | main3 / 08fa0a3 | Sous-étape 2b : pile PME générée (serveur, témoin), amorçage etcd, secrets locaux, HAProxy, images épinglées ; essai sur trois machines réussi ; procédure du banc |
| 1.7 | 2026-10-29 | main3 | Sous-étape 2c : nœud Rust recentré (garde de quorum en lecture, époque = timeline, fencing continu) ; méthode de travail : étapes enchaînées, commit par étape sans push |
| 1.8 | 2026-10-29 | main3 | Sous-étape 2d : mode synchrone selon la taille du cluster, alerte « sous le seuil », mesures ; ADR-006 ; essai sur trois machines 13/13 |
| 1.9 | 2026-10-29 | main3 | Étape 3 : le nœud héberge une application externe (manifeste, lanceur, base dédiée, reverse proxy) ; ADR-007 |
| 2.0 | 2026-10-29 | main3 | Étape 4 : réseau de l'application sans sortie, enrôlement des serveurs par code à usage unique, sauvegardes chiffrées sur le relais, chiffrement du disque vérifié ; ADR-008 ; routes de blobs du relais corrigées |

# 1. Objet de la refonte

## 1.1 Ce qu'est une « enveloppe d'exécution »

Jusqu'en V2, AMANE embarquait sa propre application métier (gestion de stock) au-dessus de son socle de souveraineté. La V3 inverse la logique : **AMANE devient l'enveloppe dans laquelle s'exécute un SaaS édité par un tiers**.

- Un éditeur externe soumet l'**exécutable** de son SaaS à AMANE (en priorité une image Docker).
- AMANE le déploie sur le cluster de la PME et en assure l'**orchestration** : haute disponibilité, réplication, bascule, réseau, sauvegarde.
- Le SaaS s'exécute **sur la base de l'architecture AMANE** : ses données restent chez la PME et ne sortent que chiffrées.
- L'**interface utilisateur et la couche métier du SaaS ne changent pas**. L'utilisateur final retrouve exactement l'application qu'il connaît ; l'éditeur externe n'a pas à réécrire son code ni à intégrer un SDK.

## 1.2 La promesse à tenir

> **Personne d'autre que la PME ne voit de données en clair. La DEK et toutes les clés de chiffrement restent dans le cluster PME. Ni l'éditeur AMANE, ni le relais, ni l'éditeur du SaaS externe n'ont accès au clair.**

Cette promesse est le cœur d'AMANE ; toute décision de la refonte est évaluée à son aune.

# 2. État actuel du dépôt (V2, point de départ)

## 2.1 Point de départ figé

- Branche de travail créée : `main3`, à partir de `main2` (commit `e0f6d09`), poussée sur `origin/main3`.
- Le travail V2 non commité (76 fichiers) a été figé dans le commit `6da5022` « chore: état V2 avant nettoyage ». C'est le point de retour de référence avant toute suppression.

## 2.2 Cartographie des composants

| Composant | Chemin | Rôle en V2 |
|---|---|---|
| SaaS éditeur (Django) | `config/`, `tenants/`, `licenses/`, `devices/`, `dashboard/`, `api/`, `templates/` | Comptes tenants, licences, parc installé, génération des installeurs PME |
| Console web (React) | `frontend/` | Console éditeur et espace tenant, consomme l'API DRF |
| Orchestrateur cluster (Go) | `orchestrator-go/` | etcd v3, Patroni, réplication CRDT, mesh WireGuard, serveur gRPC mTLS |
| Socle crypto et journal (Rust) | `spike/crates/ss-crypto`, `ss-journal`, `ss-consensus` | AK/DEK, CRL, Shamir, journal CBOR chiffré, époque et fencing, garde de quorum |
| SDK client métier (Rust) | `core-rust/ss-client/` | Invariants stock, mode hors-ligne, appairage, licence, client gRPC |
| Nœud PME (Rust) | `spike/node/` | CLI, supervision PostgreSQL, annonce au relais, **application stock et utilisateurs intégrée** |
| Relais zero-knowledge (Rust) | `spike/relay/`, `relay-deploy/` | Stockage de blobs opaques et topologie du cluster |
| Contrat gRPC | `proto/amane/framework/v1/framework.proto` | Interface unique B ↔ C (Ping, Enroll, Write, Read, NotifyRevocation, PushDelta) |
| Déploiement PME | `pme-deploy/`, `templates/tenants/*.tmpl` | PostgreSQL primaire/standby + nœud + pgAdmin + watchtower |
| CI et mesures | `.github/workflows/`, `.ci/`, `scripts/`, `tests/`, `docs/bench/` | Build, couverture, chaos, contrats, benchmarks, mesures de bascule |

## 2.3 Trois constats qui orientent le tri

**Constat 1 — Deux mécanismes de haute disponibilité coexistent.** L'orchestrateur Go (etcd + Patroni) n'est utilisé que dans le `docker-compose.yml` de développement et dans la CI. La pile réellement déployée chez la PME n'en contient pas : elle repose sur une réplication streaming PostgreSQL nue et sur le nœud Rust, qui décide seul de la promotion d'un standby (`spike/crates/ss-consensus/src/quorum.rs`, élection déterministe par LSN). Cette élection écrite à la main est en tension avec la règle n° 2 du projet (« aucun algorithme de consensus écrit à la main »).

**Constat 2 — Le nœud PME embarque sa propre application métier.** Le module `spike/node/src/stock/` et les pages HTML associées (articles, utilisateurs, tableau de bord) constituent une application métier complète. C'est précisément le rôle que tiendra désormais le SaaS externe.

**Constat 3 — La console éditeur existe en double.** Une version rendue côté serveur par Django (`templates/*.html`, `views_web.py`, `views.py`) et une version React (`frontend/`) couvrent les mêmes

écrans.

Autres observations : le nœud déclare la dépendance `ss-client` dans le workspace mais ne l'utilise pas ; un fichier proto V1 (`proto/framework.proto`) déclare le même paquet que le contrat qui fait autorité ; plusieurs scripts et notes isolés traînent à la racine.

# 3. Décisions d'architecture

Chaque décision est consignée sous forme d'ADR (Architecture Decision Record) : contexte, décision, conséquences. Une décision n'est jamais effacée : si elle est remplacée, elle passe au statut « Remplacée par ADR-xxx ».

| ADR | Titre | Statut | Date |
|---|---|---|---|
| ADR-001 | Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go | Acceptée | 2026-10-28 |
| ADR-002 | Formats d'exécutables acceptés : Docker d'abord, binaires et .exe ensuite | Acceptée | 2026-10-28 |
| ADR-003 | Zero-knowledge strict : aucun clair hors du cluster PME, clés confinées | Acceptée | 2026-10-28 |
| ADR-004 | Modalités de la HA PME : témoin etcd, synchrone strict dès 3 nœuds, garde de quorum en lecture | Acceptée | 2026-10-28 |
| ADR-005 | Installation de la pile HA chez la PME : amorçage etcd, témoin, images, secrets, HAProxy | Acceptée | 2026-10-28 |
| ADR-006 | Réglages de bascule : taille du cluster retenue, `loop_wait`, cibles de délai | Acceptée (décidée seule, méthode du 29/10) | 2026-10-29 |
| ADR-007 | Hébergement de l'application externe : lancement, interface, base, injection des identifiants | Acceptée (décidée seule) | 2026-10-29 |
| ADR-008 | Confinement et chiffrement : réseau sans sortie, enrôlement, sauvegardes, chiffrement au repos, mesh reporté | Acceptée (décidée seule) | 2026-10-29 |

## ADR-001 — Haute disponibilité PME : etcd + Patroni piloté par l'orchestrateur Go

**Contexte.** Deux mécanismes de HA coexistent (constat 1). La garde de quorum Rust (`quorum.rs`) réalise une élection de primaire écrite à la main, ce que la règle n° 2 interdit. L'orchestrateur Go, lui, s'appuie sur etcd (Raft) et Patroni, est déjà couvert par les tests d'intégration et par les mesures de bascule de la CI (switchover ~2,2 s, failover ~3,2 s).

**Décision.** La pile PME adopte **etcd + Patroni, piloté par l'orchestrateur Go**. C'est le seul mécanisme de bascule et de fencing en production.

**Conséquences.**

- Les compose PME (`pme-deploy/`, `templates/tenants/docker-compose.yml.tmpl`) intègrent etcd, Patroni (spilo) et l'orchestrateur Go, à l'image du compose de développement.
- Le nœud Rust ne promeut plus PostgreSQL lui-même (`run_supervision_loop`, sonde des pairs, `cmd/failover.rs`, `promote_standby` retirés). `quorum.rs` n'est pas supprimé mais **réorienté** en garde de quorum en lecture (voir ADR-004).
- La règle « ≥ 3 nœuds → failover automatique par quorum » est portée par etcd/Patroni. Le cas des clusters à 2 serveurs PostgreSQL est tranché par ADR-004 (témoin etcd).
- L'époque et le fencing (`epoch.rs`, `fencing.rs`) restent dans le socle ; leur source et leur contrôle continu sont fixés par ADR-004.

- **Réplication synchrone effective.** La pile V2 positionne `synchronous_commit=remote_write` sans définir `synchronous_standby_names` : PostgreSQL valide alors chaque transaction sur la seule écriture locale, la réplication est en réalité asynchrone. Cela contredit la règle « synchrone pour les invariants forts, blocage si aucun passif n'est joignable ». L'étape 2 la rend effective via Patroni (`synchronous_mode` et `synchronous_mode_strict`), qui bloque les écritures plutôt que de dégrader silencieusement en asynchrone (décision du 2026-10-28).

## ADR-002 — Formats d'exécutables acceptés

**Contexte.** Un SaaS externe peut être livré sous plusieurs formes : image Docker, binaire Linux, exécutable Windows (.exe). Supporter tous les formats d'emblée disperserait l'effort.

**Décision.** AMANE accueillera **plusieurs types d'exécutables**. L'effort porte d'abord **exclusivement sur les images Docker**. Les binaires et les .exe seront développés ensuite.

**Conséquences.**

- Le format de description d'une application soumise (le « manifeste ») porte dès le départ un champ **type d'artefact** (`docker`, puis `binaire`, `exe`), afin que l'ajout ultérieur des autres formats n'oblige pas à casser le modèle de données ni le contrat.
- Les images sont référencées par **digest** (`sha256:…`) et non par tag, pour que la PME exécute exactement ce qui a été soumis.
- Piste pour la suite (non tranchée) : envelopper un binaire nu dans une image de base fournie par AMANE ; un .exe Windows demandera des conteneurs Windows ou une machine virtuelle.

## ADR-003 — Zero-knowledge strict

**Contexte.** La promesse d'AMANE est que seule la PME voit ses données en clair. En V3 un acteur nouveau entre dans le périmètre : l'éditeur du SaaS externe, dont le code s'exécute **à l'intérieur** du cluster PME et manipule donc du clair.

**Décision.** Le zero-knowledge est maintenu sans compromis : **personne d'autre que la PME ne voit de clair**, et **la DEK comme toutes les clés de chiffrement restent dans le cluster PME**.

**Conséquences.**

- **Confinement réseau** : l'application externe s'exécute sur un réseau interne sans sortie vers Internet par défaut. Seuls sont exposés son interface utilisateur (via le reverse proxy du nœud) et sa base de données gérée par AMANE. Elle ne peut pas renvoyer de données en clair à son éditeur.
- **Chiffrement au repos** : les données de l'application (base, volumes) sont chiffrées sous des clés dérivées de la hiérarchie AMANE ; la DEK ne quitte jamais le cluster.
- **Sortie uniquement chiffrée** : sauvegardes, réplication inter-sites et tout transit par le relais sont chiffrés côté PME avant envoi. Le relais ne voit que des blobs opaques.
- **Télémétrie** : le SaaS éditeur AMANE ne reçoit que des métadonnées (état du cluster, licence, santé), jamais de contenu métier.
- **Aucun composant hors PME** (Django, relais, éditeur externe, agent IA) ne touche AK/DEK. Les opérations sur les clés restent humaines.

## ADR-004 — Modalités de la haute disponibilité PME

**Contexte.** ADR-001 confie la HA à etcd + Patroni + orchestrateur Go. Sept points restaient à trancher pour la pile PME réelle, répartie sur plusieurs machines. Le plus structurant : etcd n'opère qu'avec une majorité de ses membres ; avec deux membres, la perte de **l'une ou l'autre** machine arrête etcd, et Patroni repasse alors le primaire en lecture seule par sécurité. Une question de fond a aussi été posée : retirer l'élection du nœud Rust fait-il perdre la garantie de quorum ?

**Décisions (2026-10-28).**

| # | Sujet | Décision |
|---|---|---|
| 1 | Clusters à 2 serveurs PostgreSQL | **Témoin etcd sur une 3e machine de la PME** : etcd seul, sans PostgreSQL ni données. La majorité etcd survit à la perte d'une machine ; la bascule automatique devient possible ; tout reste chez la PME |
| 2 | Réplication synchrone | **Stricte à partir de 3 serveurs PostgreSQL** (écritures bloquées si aucun standby synchrone) ; **en dessous, non stricte avec alerte visible** dans la console : le cluster continue d'écrire quand son standby tombe, mais la PME est prévenue qu'elle n'est plus protégée |
| 3 | Version PostgreSQL | **17** (image Spilo), une seule version à maintenir |
| 4 | Clusters déjà installés | **Réinstallation des bancs** (Pilote AMANE, AMANE Test) ; pas de migration de données |
| 5 | Accès au primaire | **HAProxy local** sur chaque machine, qui interroge Patroni (`GET /primary`) : adresse fixe pour le nœud et pour l'application externe |
| 6 | Chiffrement du trafic entre machines | **Reporté à l'étape 4** (mesh WireGuard, `orchestrator-go/mesh`) ; ce trafic reste dans le périmètre PME. *Reporté de nouveau au-delà de l'étape 7 : voir ADR-008, décision 5* |
| 7 | Source de l'époque | **Timeline PostgreSQL** : incrémentée à chaque promotion, persistée, identique sur tous les nœuds, ne recule jamais |

**Le quorum reste une garantie du socle.** Ce qui sort du nœud Rust, c'est la **décision** de promouvoir (désormais prise par Raft via etcd, et appliquée par Patroni). Ce qui reste dans le socle et dans l'image du nœud, c'est la **vérification** et l'**application** :

- `quorum.rs` est réorienté en **garde de quorum en lecture** : une fonction pure classe l'état lu dans etcd/Patroni en « sain », « sous le seuil » ou « quorum perdu ». Elle reste testable unitairement ; elle ne promeut plus rien.
- `epoch.rs` et `fencing.rs` sont conservés ; l'époque est la timeline PostgreSQL.
- Le nœud contrôle son époque **en continu** (et non plus au seul démarrage) : époque dépassée → plus d'écriture dans `ss-journal`, plus d'annonce comme primaire ; cluster sous le seuil → signalement au portail.

**Limites de l'existant que cette décision corrige.** L'élection de `quorum.rs` reposait sur les sondes de chaque nœud (des partitions asymétriques donnent des vues divergentes), ne persistait aucune décision, tirait l'époque d'un compteur local vérifié au seul démarrage, et ne rétrogradait jamais un primaire isolé.

**Preuves attendues.** Chaque promesse est vérifiée par un scénario de chaos automatisé sur un laboratoire simulant trois machines (sous-étape 2a) : arrêt brutal du primaire, primaire isolé (doit cesser d'écrire), partition minoritaire (aucune promotion), cluster 2 + témoin, perte d'un standby en mode strict et non strict.

## ADR-005 — Installation de la pile HA chez la PME

**Contexte.** La sous-étape 2b remplace la pile PME V2 (PostgreSQL nu, bascule décidée par le nœud Rust) par la pile validée en laboratoire (ADR-004). Il faut décider comment elle s'installe sur des machines réelles. En préparant ce travail, un défaut de zero-knowledge a été relevé : les mots de passe PostgreSQL et pgAdmin d'une PME sont dérivés de son jeton d'enregistrement (`_derive_credentials`), que le portail de l'éditeur connaît ; l'éditeur peut donc les recalculer, et l'espace PME du portail les affiche. C'est contraire à ADR-003.

**Décisions (2026-10-28).**

| # | Sujet | Décision |
|---|---|---|
| 1 | Formation du cluster etcd | Un **conteneur d'initialisation** exécuté une fois au démarrage, écrit en Go dans l'image de l'orchestrateur : il demande au portail les IP des machines déjà installées, se fait ajouter au cluster etcd **directement sur le réseau local** (`member add`) et écrit la configuration d'etcd. Logique testable, identique sous Windows et Linux ; installeurs allégés |
| 2 | Installation d'un témoin | Un **installeur « témoin » distinct**, téléchargeable depuis l'espace PME ; le portail le propose quand le cluster compte 2 serveurs |
| 3 | Provenance des images | Toutes servies par le **registre de l'éditeur** et **épinglées par digest** (`sha256:…`) : la PME exécute exactement ce qui a été validé |
| 4 | Mots de passe de la base | **Générés aléatoirement sur la première machine** et transmis aux suivantes par etcd, donc uniquement sur le réseau de la PME. L'éditeur ne peut plus les connaître ; le portail cesse d'afficher les identifiants pgAdmin, visibles uniquement en local |
| 5 | Configuration de HAProxy | **Régénérée et rechargée par l'orchestrateur** à partir des membres Patroni enregistrés dans etcd |

**Conséquences.**

- Le portail ne fournit que des métadonnées (IP des machines) ; l'amorçage etcd, les secrets et le routage vers le primaire se font sur le réseau de la PME.
- Sécurité de l'amorçage : une machine qui connaît des pairs mais ne parvient à en joindre aucun **refuse de créer un cluster séparé** (sinon deux clusters indépendants, donc deux primaires). Créer un nouveau cluster malgré des pairs connus exige une action explicite.
- Tant que le nœud Rust n'est pas recentré (2c), sa bascule automatique est neutralisée par configuration (`CLUSTER_PEERS` vide) : deux décideurs simultanés provoqueraient un double primaire.
- Les secrets transitent en clair entre machines de la PME jusqu'au mesh WireGuard (étape 4, ADR-004 décision 6) ; ils ne sortent jamais du réseau de la PME.

## ADR-006 — Réglages de bascule

**Contexte.** La sous-étape 2d devait rendre la réplication synchrone effective selon la taille du cluster (ADR-004, décision 2) et trancher les délais de bascule laissés ouverts en 2a. Décision prise seule, selon la méthode convenue le 29/10 : option recommandée retenue, options écartées consignées.

**Décisions.**

| # | Sujet | Décision | Options écartées |
|---|---|---|---|
| 1 | Taille du cluster pour le mode strict | Le **maximum de serveurs observés**, mémorisé dans etcd (`/amane/config/serveurs_installes`). Seul l'orchestrateur élu (lease etcd) applique `synchronous_mode_strict` par l'API Patroni | Le nombre de serveurs vivants : la panne d'un serveur dans un cluster de 3 aurait retiré la protection au moment où elle compte le plus |
| 2 | `loop_wait` de Patroni | **Conservé à 2 s** | 1 s : mesuré en laboratoire (10/10 scénarios), gain ≤ 1 s sur la bascule et ~1,3 s sur la mise hors service d'un primaire isolé (qui n'acquitte de toute façon aucune écriture), pour une charge d'écriture etcd doublée |
| 3 | Cibles de délai | **< 5 s** pour un crash de processus, **≈ 5 s** pour la perte d'une machine, **< 8 s** pour une partition. La partition est volontairement plus lente : la garde anti-partition attend l'expiration du heartbeat avant de forcer | Abaisser le heartbeat ou les confirmations : risque de bascule forcée sur une simple coupure réseau |

**Conséquences.** Retirer définitivement un serveur du cluster suppose d'abaisser la valeur mémorisée (`etcdctl put /amane/config/serveurs_installes <n>`) ; c'est une opération d'administration humaine, documentée ici.

## ADR-007 — Hébergement de l'application externe

**Contexte.** L'étape 3 fait du nœud l'hôte d'exécution d'un SaaS externe livré en image Docker (ADR-002), sans modifier son interface ni sa couche métier. Décisions prises seules (méthode du 29/10).

| # | Sujet | Décision | Options écartées |
|---|---|---|---|
| 1 | Lancement | **Service déclaratif** `amane-application` dans la pile générée, image épinglée par digest ; Docker redémarre le conteneur | Le nœud pilotant Docker par son socket : équivaut à lui donner les droits root de la machine |
| 2 | Interface utilisateur | Le nœud **relaie** l'application à la racine de son port web (reverse proxy, réponses en flux) ; sa propre console passe sous `/amane/` | Publier directement le port de l'application : contournerait le nœud et tout contrôle futur (appairage d'appareil, ADR-003) |
| 3 | Base de données | L'orchestrateur crée un rôle `application` et sa base (dont il est propriétaire), mot de passe généré et partagé par etcd ; l'application rejoint sa base par HAProxy (toujours le primaire) | Base partagée avec le nœud ; rôle superutilisateur |
| 4 | Injection des identifiants | Un **lanceur statique** (le binaire de l'orchestrateur, copié dans un volume par l'amorçage) est le point d'entrée du conteneur : il lit les identifiants, compose les variables décrites par le manifeste (`DATABASE_URL`…), puis exécute la commande d'origine de l'image. Aucune image n'est modifiée ; aucun mot de passe n'apparaît dans la configuration du conteneur | Variables dans le compose (le portail connaîtrait le mot de passe) ; `env_file` (lu par Compose avant que les secrets n'existent) ; reconstruire l'image de l'éditeur |
| 5 | Manifeste | JSON validé par le portail (`tenants/manifeste.py`) : type d'artefact, image par digest, commande d'origine, port, chemin de santé, variables, modèles de variables de base, volumes. `binaire` et `exe` sont refusés pour l'instant (ADR-002) | Déduire tout de l'image (commande, port) : fragile et opaque |

**Conséquences.** L'application doit savoir lire sa base depuis une variable d'environnement (cas usuel des SaaS). Les WebSockets ne sont pas encore relayés. La sécurité du relais (contrôle d'appareil appairé devant l'interface) reste ouverte (ADR-003).

## ADR-008 — Confinement et chiffrement

**Contexte.** L'étape 4 met en œuvre ADR-003 : le code de l'éditeur tourne dans le cluster et manipule du clair. Il ne doit rien pouvoir envoyer dehors, la DEK doit atteindre chaque serveur sans circuler en clair, et tout ce qui sort (sauvegardes) doit être chiffré côté PME. Décisions prises seules (méthode du 29/10).

| # | Sujet | Décision | Options écartées |
|---|---|---|---|
| 1 | Confinement réseau | L'application est seule sur un réseau Docker **interne** (`internal: true`, aucune route vers l'extérieur). Seuls le nœud (qui relaie son interface) et HAProxy (qui mène à sa base) y sont attachés ; etcd, PostgreSQL, l'orchestrateur et pgAdmin n'y sont pas | Pare-feu `iptables` ou proxy sortant filtrant (exige `NET_ADMIN`, inopérant sous Docker Desktop) ; réseau `none` (l'application ne joindrait plus sa base) |
| 2 | Enrôlement des serveurs suivants | **Opération humaine des deux côtés.** Un administrateur génère, dans la console d'un serveur qui détient la DEK (`/amane/admin/enrolement`), un code à usage unique (16 caractères, 15 minutes, tous les codes annulés après 5 échecs). Sur le nouveau serveur, il lance `ss-node enroler --depuis … --code …`. La DEK lui est renvoyée **scellée pour sa clé publique** (`DevicePublicKey::seal_dek`, `ss-crypto`) ; il vérifie qu'elle s'ouvre avant de l'enregistrer | DEK partagée par etcd (en clair sur le réseau PME, lisible par le témoin) ; parts Shamir à chaque ajout (lourd pour une PME) ; transmission automatique à tout nouveau membre (contraire à la règle « les opérations sur les clés restent humaines ») |
| 3 | Sauvegardes | Le **primaire autorisé à écrire** exporte chaque base (`pg_dump`, format custom), chiffre l'export avec la DEK et le dépose sur le relais en blob opaque. Première sauvegarde 5 minutes après la prise de rôle, puis toutes les 24 h ; rotation sur 7 jours par la clé du blob (`sauvegarde-<base>-<jour>`) ; chaque dépôt est consigné dans le journal du nœud. `ss-node sauvegarder` force une sauvegarde ; `ss-node restaurer --base … --jour …` restaure (opération humaine) | pgBackRest ou WAL-G vers un stockage objet (chiffrement par mot de passe hors de la hiérarchie AMANE, dépendance à un cloud) ; sauvegarde orchestrée par le portail (il verrait du clair) |
| 4 | Chiffrement au repos | **Délégué au chiffrement du disque** de chaque machine (BitLocker sous Windows, LUKS sous Linux). L'installeur le vérifie, avertit s'il est absent et l'état remonte au portail (`Device.chiffrement_disque`), qui pourra le signaler à la PME. Non bloquant | Volume chiffré (gocryptfs, LUKS en fichier) sous clé dérivée de la DEK : FUSE ou privilèges impossibles sous Docker Desktop, et le déverrouillage à chaque redémarrage exigerait une opération humaine sur la clé ; TDE : absent de PostgreSQL |
| 5 | Chiffrement entre machines (mesh WireGuard) | **Reporté au-delà de l'étape 7** (modifie ADR-004, décision 6). Ce trafic reste sur le réseau local de la PME ; la DEK n'y circule jamais en clair (décision 2) et les sauvegardes sortent chiffrées (décision 3) | Mesh dès maintenant : exige `NET_ADMIN` et le module noyau WireGuard dans les conteneurs, non garanti sous Docker Desktop (WSL2) ; à valider sur le banc d'abord |
| 6 | Taille des blobs du relais | Limite portée à 1 Gio par blob (2 Mo par défaut dans axum) | Découpage en morceaux : prématuré pour la taille des bases d'une PME |

**Conséquences.** Une application qui appellerait une API externe (licence, télémétrie, paiement) ne fonctionnera pas : c'est voulu, et la question de la licence de l'application reste ouverte (§ 7). Un serveur ajouté sans enrôlement fonctionne (réplication, bascule), mais ne peut ni chiffrer son journal ni sauvegarder ; s'il devient primaire, il le signale dans ses journaux. La promesse « chiffré au repos » dépend de la PME : AMANE la vérifie et la signale, sans pouvoir l'imposer.

# 4. Tri du dépôt

## 4.1 À garder intact

| Composant | Justification |
|---|---|
| `spike/crates/ss-crypto` (AK, DEK, CRL, Shamir, device_key, recovery) | Socle de la souveraineté, indépendant de tout métier |
| `spike/crates/ss-journal` | Journal CBOR chiffré append-only, générique |
| `ss-consensus/src/epoch.rs`, `fencing.rs` | Anti split-brain générique (lease etcd) |
| `spike/relay/`, `relay-deploy/` | Le relais zero-knowledge convient tel quel |
| `proto/amane/framework/v1/`, `tests/contracts/`, `buf.gen.yaml` | Contrat figé : on pourra ajouter des messages, jamais modifier l'existant |
| `orchestrator-go/` (consensus, supervisor, grpcserver, mesh, replication, telemetry) | Devient le moteur HA de production (ADR-001) |
| Modèles `Tenant`, `License`, `Device` ; `devices/api_register.py` ; `api/` | Comptes, licences et identité d'installation inchangés |
| `.github/workflows/`, `.ci/`, `scripts/*_measure.sh`, `docs/bench/` | Garde-fous CI et mesures de bascule |

## 4.2 À garder mais adapter

| Composant | Adaptation prévue |
|---|---|
| `spike/node` | Devient l'**hôte d'exécution** : garde init, run, status, delist, le client relais et le heartbeat SaaS ; gagne le lancement et la supervision de l'application externe à partir de son manifeste, la base PostgreSQL gérée, et un reverse proxy qui sert l'interface de l'application telle quelle |
| `core-rust/ss-client` | N'est plus un SDK intégré par l'application (elle reste inchangée). `license.rs`, `pairing.rs` et `offline.rs` sont rapatriés dans le nœud ou conservés comme bibliothèque interne |
| Django (`tenants/`, `licenses/`, `devices/`, `dashboard/`) | Nouveau modèle **Application** (image soumise, digest, manifeste : ports, volumes, base requise, variables), rattaché à la licence ; génération des installeurs et du compose incluant l'application |
| `templates/tenants/*.tmpl`, `pme-deploy/` | Ajout du service applicatif et de la pile etcd/Patroni/orchestrateur (ADR-001) ; `watchtower` revu (mise à jour par digest vérifié uniquement) ; `pgadmin` rendu optionnel |
| `frontend/` | Garde Tenants, Licences, Appareils, Clusters, Relais, MonEspace, Login, Register, Déploiement ; ajoute une page Applications ; met à jour la page Architecture |
| `ss-consensus/src/supervision.rs` | **Supprimé (2c)** : le nœud lit Patroni ; `ss-consensus` ne dépend plus de `sqlx` |
| `ss-consensus/src/quorum.rs` | **Reclassé (ADR-004)** : initialement à supprimer, il est réorienté de l'élection vers une garde de quorum en lecture (sain / sous le seuil / perdu) |

| Composant | Adaptation prévue |
|---|---|
| `README.md`, `CLAUDE.md`, `ACCES-INTERFACES.md`, `CONTRAT-API.md`, `Architecture_SaaS_Souverain_Trois_Acteurs.md` | Réécrits une fois la refonte stabilisée |

## 4.3 À supprimer

| Élément | Raison |
|---|---|
| `spike/node/src/stock/` et les routes HTML liées au stock | Métier AMANE remplacé par le SaaS externe |
| `core-rust/ss-client/src/invariants.rs` | Invariants propres au stock |
| `frontend/src/pages/StockPage.tsx` | Écran du métier supprimé |
| `proto/framework.proto` | Vestige V1, même paquet que le contrat qui fait autorité |
| Console Django en double : `templates/**/*.html`, `views_web.py`, `views_espace.py`, `dashboard/views*.py`, `urls.py` web, `static/` | Doublon de la console React (couverture vérifiée écran par écran). Les `.tmpl` (générateurs d'installeurs) sont conservés. **Correction** : les `views.py` de `tenants/`, `licenses/` et `devices/` sont les ViewSets DRF de l'API et restent en place |
| `spike/node/Dockerfile.stub`, `.gitkeep` inutiles | Vestiges |
| `fix_locks.py`, `verify.py`, `docker-cleanup.ps1` | Scripts ponctuels ; `fix_locks.py` contient un mot de passe en clair |
| `.agenthub/sessions/` | Sans rapport avec le projet |
| `paroles_explication_mission_b.txt` | Note personnelle |
| `impact_missions_v1_vers_v2 (1).md`, `Stack_Technique_Framework_Souverainete_SaaS (1).md`, `schema_soutenance.html` | Déplacés vers `docs/archive/` plutôt que supprimés (le schéma de soutenance n'a pas d'équivalent React) |

# 5. Plan de travail par étapes

| Étape | Contenu | Statut |
|---|---|---|
| 0 | Branche `main3`, instantané V2, présent document | Terminée |
| 1 | Nettoyage : suppressions du § 4.3, un commit par catégorie | Terminée |
| 2a | Image Docker de l'orchestrateur ; laboratoire simulant trois machines ; scénarios de chaos prouvant chaque promesse (ADR-001, ADR-004) | Terminée |
| 2b | Gabarit compose PME (etcd, Patroni, orchestrateur, HAProxy), installeurs, génération par le portail, tests Django (ADR-005) | Réalisée ; validation sur le banc AMANE Test à faire (annexe) |
| 2c | Recentrage du nœud Rust : retrait de la promotion, garde de quorum en lecture, époque = timeline, fencing continu, séparation de `NODE_MODE` | Terminée |
| 2d | Réplication synchrone et alerte sous le seuil, mesures de bascule sur le banc réinstallé | Terminée (mesures en laboratoire ; banc à faire) |
| 3 | Nœud → hôte d'exécution : manifeste, lancement et supervision d'une image Docker, reverse proxy de l'interface (ADR-002, ADR-007) | Terminée |
| 4 | Confinement et chiffrement : réseau interne sans sortie, enrôlement des serveurs, sauvegardes chiffrées via le relais, chiffrement du disque vérifié (ADR-003, ADR-008) | Terminée |
| 5 | SaaS éditeur : modèle Application, soumission d'image par digest, génération des installeurs | À faire |
| 6 | Console React : page Applications, mise à jour Architecture | À faire |
| 7 | Documentation : README, CLAUDE.md, accès, contrat d'API | À faire |
| Plus tard | Binaires Linux et .exe Windows (ADR-002) | Prévu |

# 6. Journal des étapes

## Étape 0 — Préparation (2026-10-28)

- Création de la branche `main3` depuis `main2` (`e0f6d09`) et publication sur `origin/main3`.
- Commit `6da5022` « chore: état V2 avant nettoyage » : 76 fichiers V2 jusque-là non suivis ou non commités (pages React clusters, relais, espace tenant ; API DRF clusters, relais, statistiques, images ; quorum `ss-consensus` ; documents d'accès et de déploiement).
- Inventaire du dépôt et tri initial (§ 4).
- Décisions ADR-001, ADR-002 et ADR-003 actées.
- Création du présent document et de son générateur PDF.

## Étape 1 — Nettoyage (2026-10-28)

Quatre commits, un par catégorie, pour que chacun reste lisible et réversible.

| Commit | Contenu |
|---|---|
| `7ca8e78` | Racine : retrait de `fix_locks.py` (mot de passe en clair), `verify.py`, `docker-cleanup.ps1`, `.agenthub/`, d'une note personnelle ; deux notes de cadrage archivées dans `docs/archive/` |
| `81d7081` | Vestiges : `proto/framework.proto` (V1), `spike/node/Dockerfile.stub`, `.gitkeep` inutiles, dossier `infra/` vide. `CLAUDE.md` et le serveur gRPC pointent vers le seul proto restant |
| `5609fb6` | Métier stock : module `spike/node/src/stock/`, `ss-client/src/invariants.rs`, `StockPage.tsx` (routée nulle part). La racine `/` du nœud affiche une page « Application hébergée », futur point d'entrée du reverse proxy (étape 3). Authentification et console cluster du nœud conservées |
| `dc06197` | Console Django : vues et gabarits HTML retirés ; les fonctions partagées avec l'API passent dans des modules `services.py` (historique conservé), la voie PME par jeton dans `tenants/api_pme.py`. Aucune URL d'API ne change |

**Choix faits pendant l'étape.**

- `quorum.rs` reste en place jusqu'à l'étape 2 : le retirer avant qu'etcd/Patroni n'arrivent dans la pile PME supprimerait toute bascule automatique chez les clients.
- Les tables `articles` et `mouvements_stock` déjà créées dans les bases PME ne sont pas supprimées : le nœud ne les crée plus, mais effacer des données client n'est pas du ressort d'un nettoyage de code.
- Le schéma de soutenance, sans équivalent React, est archivé plutôt que supprimé.

**Vérifications.**

- Django : `manage.py check` sans erreur ; 75 tests exécutés sur SQLite en mémoire, 74 réussis. L'échec (`test_options_de_replication_postgres`) existait déjà avant le nettoyage (vérifié sur le commit `098f1de`), voir § 7.
- Frontend : `npm run build` et `npm run lint` sans erreur.
- Go : `go build ./...` sans erreur.
- Rust : **non compilé** pendant l'étape (Docker arrêté sur le poste, aucun toolchain local) ; diff relu à la main. Compilation confirmée ensuite, voir les compléments ci-dessous.

## Étape 1 — Compléments (2026-10-28)

**Compilation Rust (via Docker, `rust:1.88-bookworm`).**

- `spike/` : compile, 74 tests réussis, 0 échec.
- `core-rust/ss-client` : compile, 7 tests unitaires réussis.
- Les avertissements restants (`delete_session` jamais appelée, champ `node_id` jamais lu, variable `e` inutilisée dans `ss-journal`) existaient avant le nettoyage ; ceux du module stock ont disparu avec lui.

**Test d'intégration `core-rust/ss-client/tests/mock_server.rs` retiré.** Il implémentait le contrat V1 (`RegisterNode`, `WriteOperation`, `GetClusterStatus`) alors que `build.rs` génère les bindings depuis le contrat V2 : 25 erreurs de compilation, déjà présentes avant le nettoyage (vérifié sur `098f1de`). Il empêchait `cargo test` de lancer les tests unitaires de `ss-client`. Une version alignée sur le contrat V2 pourra être écrite si le besoin revient.

**Test `test_options_de_replication_postgres` corrigé.** Le test attendait `max_wal_senders=3` et `wal_keep_size=64` ; le gabarit compose (comme `pme-deploy/`) utilise 5 et 512. Le second écart était masqué par l'arrêt du test au premier échec. Les valeurs du gabarit font foi :

- `max_wal_senders=5` : deux standbys en flux et un `pg_basebackup -Xs` (qui occupe deux émetteurs) doivent pouvoir coexister sur un cluster de trois nœuds ; avec 3, le reclonage d'un standby peut être refusé.
- `wal_keep_size=512` (Mo) : un standby redémarré ou brièvement coupé rattrape son retard sans être recloné ; 64 Mo n'offrent pas cette marge.

Le test est aligné et commenté. Suite Django : 75 tests, 75 réussis.

**Constat reporté à l'étape 2.** En analysant ce test, la réplication s'est révélée asynchrone de fait (`synchronous_commit=remote_write` sans `synchronous_standby_names`). Correction décidée pour l'étape 2, consignée dans ADR-001.

## Étape 2a — Image de l'orchestrateur et laboratoire HA (2026-10-28)

**Objectif.** Avant de modifier la pile installée chez les PME (2b) et le nœud Rust (2c), prouver que l'architecture décidée par ADR-001 et ADR-004 tient ses promesses quand on la casse. La preuve est apportée par un laboratoire qui simule trois machines sur un seul poste, et par des scénarios de chaos automatisés qui rendent un verdict OK / ÉCHEC.

### Ce qui a été livré

| Fichier | Rôle |
|---|---|
| `orchestrator-go/Dockerfile`, `.dockerignore` | Image de l'orchestrateur (binaire Go statique, utilisateur non privilégié, 33 Mo). Elle n'existait pas : l'orchestrateur n'était lancé qu'à la main |
| `ha-lab/docker-compose.yml` | Laboratoire : trois « machines » simulées, deux topologies |
| `ha-lab/chaos.sh` | Les scénarios de chaos et leur bilan (topologie `temoin`, `trois-pg` ou `tout`, filtre de scénario optionnel) |
| `ha-lab/README.md` | Notice du laboratoire |
| `orchestrator-go/supervisor/beacon.go`, `supervisor.go`, `consensus/client.go` | Trois défauts corrigés (voir plus bas) |

| Fichier | Rôle |
|---|---|
| `orchestrator-go/supervisor/beacon_integration_test.go` | Test de non-régression du défaut de heartbeat |
| `.ci/testcontainers/main.go`, `.github/workflows/build-go.yml`, `chaos-tests.yml` | etcd de CI réglé comme en production ; nouveau test ajouté à la campagne de chaos CI |

### Le laboratoire

Chaque « machine » `mN` est un groupe de conteneurs relié à deux réseaux : `lan`, qui joue le réseau local de la PME, et `mN`, qui joue le bus interne de la machine. Débrancher les conteneurs d'une machine du réseau `lan` simule un câble réseau arraché : ils continuent de se voir entre eux, mais plus les autres machines. Sur `lan`, chaque conteneur a une IP fixe et les services se désignent par IP, comme de vraies machines.

| Machine | etcd | PostgreSQL + Patroni | Orchestrateur |
|---|---|---|---|
| m1 | 10.77.0.11 | 10.77.0.12 | 10.77.0.13 |
| m2 | 10.77.0.21 | 10.77.0.22 | 10.77.0.23 |
| m3 | 10.77.0.31 | 10.77.0.32 (topologie trois-pg) | 10.77.0.33 (topologie trois-pg) |

Deux topologies, qui correspondent aux deux cas d'ADR-004 :

- **Topologie « témoin »** : deux serveurs PostgreSQL (m1, m2) et un témoin etcd seul sur m3 (décision 1). Réplication synchrone **non stricte** (décision 2).
- **Topologie « trois-pg »** : trois serveurs PostgreSQL. Réplication synchrone **stricte**.

Réglages Patroni communs : `ttl` 20 s, `loop_wait` 2 s, `retry_timeout` 5 s, `synchronous_mode` activé. Chaque topologie est recréée à neuf avant ses scénarios.

### Comment les pannes sont simulées

| Panne réelle | Simulation dans le laboratoire |
|---|---|
| PostgreSQL ou Patroni plante, la machine reste en vie | `docker kill -s KILL` du seul conteneur PostgreSQL ; l'etcd et l'orchestrateur de la machine continuent de tourner |
| La machine s'éteint brutalement (coupure de courant) | `docker kill -s KILL` de tous les conteneurs de la machine |
| Câble réseau arraché, switch défaillant | Débranchement de tous les conteneurs de la machine du réseau `lan` ; ils restent vivants |

### Comment le verdict est rendu

- **Écriture acquittée** : un `INSERT` est acquitté si PostgreSQL confirme le `COMMIT` en moins de 5 s. En réplication synchrone, ce `COMMIT` n'est confirmé qu'après réception par le standby synchrone : une écriture acquittée existe donc sur deux machines.
- **Leader** : il est lu dans etcd **auprès de la majorité**. Un membre etcd isolé refuse de répondre (lecture linéarisable), on interroge alors le suivant.
- **Rétrogradation** : on vérifie que PostgreSQL n'accepte plus d'écriture (`pg_is_in_recovery()` vrai, ou PostgreSQL arrêté). Le rôle annoncé par l'API REST de Patroni ne suffit pas : un primaire privé d'etcd peut encore renvoyer son dernier rôle connu alors que Patroni a déjà arrêté PostgreSQL.

- **Zéro perte** : un marqueur est écrit et acquitté juste avant la panne ; il doit se retrouver sur le nouveau primaire.
- **Retour à la normale** : après chaque scénario, le cluster doit se reformer complètement (tous les membres présents, un leader, au moins un standby synchrone).

### Les scénarios et les promesses qu'ils prouvent

**1. Crash du PostgreSQL primaire.** On tue le seul processus PostgreSQL du primaire. On vérifie qu'un autre nœud devient primaire et acquitte des écritures, que le marqueur est présent (zéro perte), puis qu'au redémarrage l'ancien primaire revient **en réplica** et non en second primaire. *Promesse : bascule automatique sans perte, et fencing de l'ancien primaire.*

**2. Panne de la machine du primaire.** On tue toute la machine du primaire : son etcd, son PostgreSQL, son orchestrateur. On vérifie la bascule et le zéro perte. Dans la topologie témoin, c'est le témoin etcd qui fournit la majorité : sans lui, etcd s'arrêterait et Patroni bloquerait toute écriture. *Promesse : la majorité restante suffit à basculer (décision 1 d'ADR-004).*

**3. Isolement réseau du primaire.** C'est le scénario le plus exigeant : le primaire est coupé du réseau mais reste vivant. Pendant qu'une écriture est tentée sur ce primaire isolé, on surveille la bascule côté majorité. On vérifie que **le primaire isolé n'acquitte aucune écriture**, que la majorité élit un nouveau primaire, que l'ancien cesse lui-même d'accepter des écritures, et qu'au retour du réseau **l'écriture non acquittée n'apparaît nulle part** dans le cluster (elle est effacée par `pg_rewind`). *Promesse : jamais deux primaires capables d'acquitter une écriture (anti split-brain).*

**4. Perte du standby, non strict** (topologie témoin). On tue la machine du standby. On vérifie que les écritures reprennent rapidement sur le primaire, désormais sans standby synchrone. *Promesse : en dessous de 3 serveurs PostgreSQL, le cluster continue d'écrire (décision 2) ; l'alerte « vous n'êtes plus protégé » est à ajouter à l'étape 2d.*

**5. Isolement du témoin etcd** (topologie témoin). On coupe le témoin du réseau pendant 15 s. On vérifie que le primaire ne change pas et que les écritures continuent. *Promesse : le témoin ne crée pas de point de défaillance.*

**6. Partition minoritaire d'un standby** (topologie trois-pg). On isole un standby pendant 30 s. On vérifie qu'il ne se promeut jamais, que le primaire reste le même et qu'il continue d'acquitter des écritures grâce à l'autre standby. *Promesse : un nœud minoritaire ne peut jamais devenir primaire.*

**7. Blocage strict sans standby** (topologie trois-pg). On tue les deux standbys. On vérifie que le primaire **n'acquitte plus aucune écriture**, puis que les écritures reprennent au retour des standbys. *Promesse : en mode strict, aucune écriture n'est confirmée si elle n'existe pas sur deux machines (décision 2).*

Les scénarios 1 à 3 sont joués sur les deux topologies : 10 exécutions au total.

### Résultats (campagne finale)

| Scénario | Témoin (2 PG + témoin) | Trois-pg (3 PG, strict) |
|---|---|---|
| 1. Crash du PostgreSQL primaire | OK — bascule en 3,7 s, zéro perte | OK — bascule en 3,3 s, zéro perte |
| 2. Panne de la machine du primaire | OK — bascule en 5,7 s, zéro perte | OK — bascule en 4,8 s, zéro perte |
| 3. Isolement réseau du primaire | OK — aucune écriture acquittée par l'isolé ; bascule en 6,3 s ; l'isolé n'accepte plus d'écriture après 9,5 s ; écriture non acquittée écartée | OK — mêmes vérifications ; bascule en 7,3 s ; isolé hors service après 9,8 s |

| Scénario | Témoin (2 PG + témoin) | Trois-pg (3 PG, strict) |
|---|---|---|
| 4. Perte du standby, non strict | OK — écritures reprises en 1,2 s | — |
| 5. Isolement du témoin | OK — aucun effet pendant 15 s | — |
| 6. Partition minoritaire | — | OK — aucune promotion en 30 s |
| 7. Blocage strict | — | OK — aucune écriture acquittée sans standby, reprise au retour |

**Toutes les promesses de sécurité sont tenues** : zéro perte, aucun double primaire, aucune promotion minoritaire, blocage strict effectif.

### Défauts trouvés et corrigés dans l'orchestrateur

Les premières campagnes ont mis au jour trois défauts réels du superviseur Go, invisibles dans les tests existants qui ne simulaient pas de partition réseau entre machines.

| Défaut | Effet constaté | Correction |
|---|---|---|
| **Client etcd sans keepalive gRPC** | Après une partition, le client restait accroché à sa connexion vers le membre etcd isolé : heartbeats, droit de forcer et libération du verrou échouaient tous. La bascule n'était plus accélérée et retombait sur le `ttl` de Patroni (21 à 26 s) | Keepalive d'une seconde (`consensus/client.go`). Le serveur etcd doit l'accepter : `--grpc-keepalive-min-time 1s` (laboratoire, CI, et pile PME à l'étape 2b) |
| **Délai de libération du verrou Patroni trop court** | La suppression du verrou dans etcd avait 300 ms (le délai d'une sonde REST) ; juste après une partition, elle échouait et toute la détection repartait de zéro | Délai d'écriture etcd dédié de 2 s (`supervisor.go`) |
| **Heartbeat jamais rétabli après une isolation** (défaut de sécurité) | Une fois reconnecté, l'orchestrateur réutilisait une lease etcd expirée et ne battait plus jamais le cœur de son nœud. Si ce nœud redevenait primaire, la garde anti-partition ne le protégeait plus : une simple coupure REST aurait pu déclencher une bascule forcée à tort | Session recréée dès que la lease est morte (`beacon.go`). Test d'intégration `TestHeartbeatSessionRecreatedAfterLeaseLoss` : **il échoue sur le code d'origine et passe avec la correction** |

Effet mesuré des corrections sur l'isolement du primaire : de 21,8 à 25,8 s avant, à 6,3 et 7,3 s après.

### Artefacts de laboratoire corrigés

Ces écarts tenaient au laboratoire ou au script de test, pas à l'architecture ; ils sont consignés parce que certains renseignent l'étape 2b.

- **Adresse annoncée par Spilo.** Spilo ignore `PATRONI_POSTGRESQL_CONNECT_ADDRESS` et lit `PG_CONNECT_ADDRESS` / `RESTAPI_CONNECT_ADDRESS`, avec pour défaut l'IP du conteneur. Après une reconnexion réseau, l'adresse annoncée était périmée. *Pour 2b : annoncer l'IP LAN de la machine.* Le compose de développement est concerné.
- **DNS d'un conteneur arrêté.** La résolution du nom d'un conteneur arrêté met environ 3 s à échouer, ce qui retardait la promotion. D'où les IP fixes, plus fidèles à une PME.
- **Rôle REST en cache.** Voir « Rétrogradation » ci-dessus.

- **Rythme des scénarios.** Voir le cooldown ci-dessous.
- **Mesure de l'isolement.** La tentative d'écriture sur le primaire isolé est menée en parallèle de la surveillance de la bascule ; menée avant, elle ajoutait 5 s à la mesure.

### Comportements à connaître

- **Cooldown de 30 s.** Après une bascule forcée, l'orchestrateur s'interdit d'en forcer une autre pendant 30 s (anti-oscillation). Une seconde panne dans cette fenêtre n'est rattrapée que par le `ttl` de Patroni (environ 20 s).
- **Rétrogradation hors ligne.** Un primaire privé d'etcd est **arrêté** par Patroni (« demoting self, offline ») et reste arrêté tant que le réseau n'est pas revenu ; il rejoint ensuite le cluster en réplica.
- **Passage en asynchrone (non strict).** Quand le standby d'un cluster à 2 serveurs tombe, Patroni retire le standby synchrone et les écritures reprennent en asynchrone. C'est le comportement voulu par la décision 2 ; l'alerte reste à construire (2d).
- **Option `synchronous_mode_stop_on_limit`.** Vérifié dans le code de Patroni 4.0.4 (image Spilo) : cette option n'existe pas et est ignorée. Seules `synchronous_mode` et `synchronous_mode_strict` existent. Le compose de développement est à corriger.

### Délais de bascule et cible de 5 s

Les crashs de processus basculent en 3,3 à 3,7 s et la panne d'une machine en 4,8 à 5,7 s, autour de la cible. L'isolement réseau prend 6,3 à 7,3 s. Ce délai a une borne basse structurelle : avant de forcer, l'orchestrateur doit constater que le heartbeat du primaire a expiré (lease de 2 s, puis 2 confirmations à 0,5 s d'intervalle). Ce délai est justement ce qui empêche de forcer une bascule sur une simple coupure REST. Ensuite, Patroni ne traite la demande qu'à son tour de boucle suivant (`loop_wait` 2 s). Deux leviers sont à évaluer à l'étape 2d, sur le banc réel : `loop_wait` à 1 s, et la durée du heartbeat. Les deux augmentent le risque de fausse détection ; la cible sera tranchée sur mesures réelles.

### Vérifications

- Go : `go build`, `go vet`, tests unitaires de tous les paquets réussis.
- Intégration sur etcd réel (harnais CI) : `consensus`, `supervisor` et `mesh` réussis, dont le nouveau test de non-régression ; ce dernier échoue sur le code d'origine.
- Chaos : 10 scénarios sur 10 réussis sur la campagne finale.
- Aucun changement Django, React ni Rust dans cette sous-étape.

## Étape 2b — Pile PME réelle (2026-10-29)

**Objectif.** Faire de la pile validée en laboratoire (2a) ce que le portail génère et ce que les installeurs déploient réellement chez une PME, selon les cinq décisions d'ADR-005.

### Comment une PME s'installe désormais

1. **Premier serveur.** L'installeur écrit un `.env` (IP LAN, nom de la machine), télécharge la pile et la démarre. L'amorçage (`orchestrator init`) demande au portail les machines déjà installées : il n'y en a pas, il crée un cluster etcd à un membre. L'orchestrateur génère les mots de passe, les enregistre dans etcd, crée le rôle `amane` sur le primaire et écrit la configuration HAProxy.

2. **Serveur suivant.** Le portail lui donne l'IP du premier. L'amorçage s'ajoute au cluster etcd **directement sur le réseau local** (`member add`), l'orchestrateur lit les mêmes mots de passe dans etcd, Patroni clone la base et en fait un standby synchrone.
3. **Témoin.** Même amorçage, puis etcd seul ; il s'annonce au portail (type `temoin`) et ne consomme pas de poste.

Plus aucune question n'est posée à l'installation (fini « IP du nœud primaire ? ») et aucun mot de passe n'est calculé par l'installeur ni connu du portail.

## Ce qui a été livré

| Domaine | Fichiers | Contenu |
|---|---|---|
| Orchestrateur Go | `bootstrap/`, `cmd/orchestrator/pme.go` | Sous-commande `init` : amorçage etcd (création, ajout, refus de créer un cluster séparé si des machines connues sont injoignables), droits des volumes, secrets générés et partagés par etcd |
| | `dcs/`, `haproxy/` | Lecture des membres et du leader Patroni ; configuration HAProxy régénérée et rechargée à chaud |
| | `provision/` | Rôle `amane` et base `amane` créés sur le primaire (idempotent) |
| | `portail/`, sous-commande `temoin` | Liste des machines, signalement du témoin (métadonnées seules) |
| | `supervisor/`, `consensus/client.go` | Découverte des membres dans etcd ; client etcd qui apprend tous les membres à partir de l'etcd local |
| Pile générée | `templates/tenants/docker-compose.yml.tmpl`, `docker-compose-temoin.yml.tmpl` | Serveur : amorçage, etcd, Spilo/Patroni 17, orchestrateur, HAProxy, nœud, pgAdmin ; témoin : amorçage, etcd, signalement |
| Installeurs | `install-linux.sh.tmpl`, `install-windows.bat.tmpl` | 8 étapes identiques ; type de machine ; arrêt de l'ancienne pile V2 sans perte de ses volumes ; nouveau dossier d'installation |
| Portail | `tenants/pile.py`, `tenants/services.py`, `api_espace.py`, `api_pme.py` | Images épinglées par digest (503 sans manifeste) ; paramètre `?type=` (serveur ou témoin) ; suppression des mots de passe dérivés ; témoin conseillé |
| | `devices/` (migration 0006) | `node_kind` ; un témoin ne consomme pas de poste |
| Console | `MonEspacePage.tsx`, types, client | Plus de mot de passe affiché ; encart et installeurs du témoin |
| Publication | `scripts/publier-pile.sh` | Pousse les six images dans le registre de l'éditeur et écrit leurs digests |
| Essais | `ha-lab/pme/` | Laboratoire d'installation machine par machine |
| Divers | `docker-compose.yml` (dev), `CONTRAT-API.md`, CI | Variables Spilo corrigées, option inexistante retirée ; contrat d'API à jour ; nouveaux paquets au plancher de couverture |

## Choix faits pendant la sous-étape

- **Rôle de connexion `amane` créé par l'orchestrateur.** Spilo ne fournit qu'un rôle `admin` sans droit de connexion (son `post_init` force `NOLOGIN`, et Patroni 4 a retiré `bootstrap.users`). L'orchestrateur, qui détient le mot de passe superutilisateur sans qu'il quitte la PME, crée le rôle et la base de façon idempotente. C'est aussi la brique dont l'étape 3 aura besoin pour la base de l'application hébergée.
- **Connexions non chiffrées admises sur le réseau PME** (`ALLOW_NOSSL`). Spilo refuse par défaut toute connexion distante sans TLS ; le nœud (sqlx sans TLS) et bien des applications ne savent pas chiffrer. Le chiffrement entre machines viendra du mesh WireGuard (étape 4, ADR-004 décision 6).
- **Le nœud reste connecté à son PostgreSQL local** (et non à HAProxy) jusqu'à son recentrage (2c) : il continue ainsi de lire le vrai rôle de sa machine. Sa bascule est neutralisée (`CLUSTER_PEERS` vide).
- `NODE_MODE` **change de sens** : il ne désigne plus le rôle PostgreSQL mais le premier serveur, qui génère la DEK. Détecté par l'installeur via `cluster-status` (les témoins ne comptent pas).
- **Watchtower retiré.** Avec des images épinglées par digest, une mise à jour passe par une nouvelle publication et une nouvelle pile, jamais par un remplacement automatique.
- **Repli sur l'archive de l'image retiré.** Une image chargée depuis une archive n'a pas le digest du registre : elle contredirait l'épinglage.
- **Réinstallation des bancs** (ADR-004 décision 4) : nouveau dossier (`/opt/amane-pme`, `%PROGRAMDATA%\Amane\PME`) ; l'ancienne pile est arrêtée mais ses volumes sont conservés.

## Défauts trouvés et corrigés en cours de route

| Défaut | Comment il a été trouvé | Correction |
|---|---|---|
| Spilo ignore `PGPASSWORD_REPLICATION` et les `PATRONI_*_PASSWORD` : le mot de passe de réplication restait `standby` | Lecture du code de Spilo | Variables lues par Spilo (`PGPASSWORD_STANDBY`…) ; compose de développement corrigé |
| Rôle `admin` de Spilo sans connexion | Laboratoire d'installation | Rôle `amane` créé par l'orchestrateur |
| Client figé derrière HAProxy après la panne d'une machine (connexion vers une machine morte jamais coupée après un rechargement) | Laboratoire d'installation | `hard-stop-after 30s`, `retries`, `option redispatch` |
| Digest d'une image multi-architecture absent du registre (« manifest unknown » chez la PME) | Essai sur trois machines | Le script de publication lit le digest auprès du registre |
| Membres Patroni nommés d'après l'identifiant du conteneur : l'orchestrateur ne reconnaissait pas son nœud | Essai sur trois machines | `hostname` du conteneur PostgreSQL = nom de la machine ; test Django dédié |
| Identifiant du témoin non inscriptible (volume root, orchestrateur non privilégié) | Relecture | Dossier préparé par l'amorçage |
| Vulnérabilité de `pgx` v5.7.5 (GO-2026-5004) et de `x/text` v0.37 rendue atteignable | `govulncheck` | `pgx` v5.9.2, `x/text` v0.41.0 |

## Vérifications

- **Go** : build, vet, tests unitaires et d'intégration sur etcd réel, tous réussis. Couverture par paquet : `dcs` 93 %, `portail` 85 %, `bootstrap` 74 %, `haproxy` 67 %, `provision` 42 % (son test d'intégration exige un PostgreSQL, absent en CI ; paquet hors plancher).

- **Django** : 71 tests réussis, dont une suite V3 de la pile (`tenants/tests.py`) : images épinglées et refus sans manifeste, absence de secret, bascule du nœud neutralisée, nom du membre Patroni, témoin, parité des installeurs, `.bat` en ASCII pur.
- **Frontend** : build et lint sans erreur.
- **Laboratoire d'installation** (`ha-lab/pme`) : 4 vérifications sur 4 (premier serveur, ajout du second avec les mêmes secrets, témoin, bascule vue par HAProxy en 6,5 s sans perte).
- **Essai Docker-dans-Docker** : trois « machines » à IP, moteur Docker et ports publiés propres, faisant tourner **le compose généré par le portail** et les images publiées par le script : premier serveur leader, second en standby synchrone, mêmes secrets, témoin à 3 membres etcd, puis coupure brutale de la machine du primaire → écritures reprises par le HAProxy de l'autre machine en 7 s, zéro perte.
- **À faire** : validation sur le banc AMANE Test (procédure en annexe) — pare-feu Windows, Docker Desktop, vraies IP LAN.

**Commits.**

| Commit | Contenu |
|---|---|
| `2722c6a` | Orchestrateur : amorçage etcd, secrets partagés, rôle `amane`, HAProxy, témoin, découverte des membres ; laboratoire `ha-lab/pme` ; CI |
| `08fa0a3` | Portail : gabarits serveur et témoin, images épinglées, suppression des secrets côté portail, installeurs, `node_kind`, console, script de publication, tests V3, compose de développement, contrat d'API |

**Suite.** La sous-étape 2c (recentrage du nœud Rust) n'est pas lancée : elle attend la validation de 2b sur le banc AMANE Test et l'accord explicite pour démarrer.

## Méthode de travail à partir du 2026-10-29

À la demande de l'utilisateur, les étapes restantes (2c à 7) sont enchaînées sans validation intermédiaire. Après chaque étape : mise à jour de ce journal, **un commit local, aucun push**. Tout point à trancher est décidé selon l'option recommandée et consigné ici sous forme d'ADR, avec les options écartées.

## Étape 2c — Recentrage du nœud Rust (2026-10-29)

**Objectif.** Appliquer ADR-004 au nœud : il ne décide plus d'aucune bascule, il **vérifie** et **fait respecter** l'état décidé par etcd et appliqué par Patroni.

**Ce qui a changé.**

| Élément | Avant | Après |
|---|---|---|
| Décision de bascule | `run_supervision_loop` : sonde des pairs (`CLUSTER_PEERS`), élection par LSN (`decider_failover`), `pg_promote` | Supprimée. Bascule manuelle : `patronictl switchover` |
| `ss-consensus/src/quorum.rs` | Élection écrite à la main | Garde de quorum en lecture : `evaluer_quorum` classe l'état Patroni en `sain`, `sous_le_seuil`, `perdu` (fonction pure, 7 tests) |
| `ss-consensus/src/supervision.rs` | Lecture de PostgreSQL et promotion | Supprimé ; `ss-consensus` ne dépend plus de `sqlx` |
| Époque | Compteur dans `config.toml`, vérifié au démarrage | Timeline PostgreSQL publiée par Patroni (`epoque_cluster`), vérifiée **à chaque tick** (5 s) |

| Élément | Avant | Après |
|---|---|---|
| Rôle annoncé au portail | `pg_is_in_recovery()` ou `NODE_MODE` | Rôle Patroni de la machine (`spike/node/src/patroni.rs`), et seulement s'il n'est pas fencé |
| `ss-node failover` | Promotion manuelle par `pg_promote` | Commande retirée |
| `NODE_MODE` | Rôle PostgreSQL **et** génération de la DEK | Uniquement « premier serveur » : génère la DEK au premier démarrage |
| Connexion PostgreSQL du nœud | PostgreSQL local | HAProxy (toujours le primaire) ; migrations idempotentes jouées par chaque nœud |
| Page « Cluster » de la console du nœud | Vues `pg_stat_*` locales | État Patroni : membres, rôles, époque, état du quorum |

**Fencing continu.** À chaque tick, une fonction pure (`decider`, 4 tests) fixe le droit d'écrire : le nœud n'écrit dans son journal en tant que primaire que si Patroni le dit primaire, que le quorum n'est pas perdu et que sa timeline est la plus récente du cluster. Un ancien primaire revenu avec une timeline inférieure est fencé, même si son PostgreSQL se croit encore primaire ; il s'annonce alors `standby`. La prise de rôle primaire est consignée dans `ss-journal` avec l'époque.

**DEK.** En mode Docker, chaque serveur générait sa propre DEK (le code le signalait comme provisoire) : les données chiffrées n'auraient pas été lisibles d'une machine à l'autre. Désormais seul le premier serveur la génère ; les suivants démarrent sans DEK, « en attente d'enrôlement », sans redémarrer en boucle (le nœud fonctionne, son journal est désactivé). L'enrôlement des serveurs suivants est traité à l'étape 4.

**Remontée au portail.** Le nœud transmet désormais l'état du cluster (`etat_quorum`) à chaque battement de cœur ; son stockage et l'alerte associée relèvent de 2d.

**Vérifications.** Workspace `spike/` : compilation et tests réussis (16 tests du nœud, 19 de `ss-consensus`, 21 de `ss-crypto`, 6 de `ss-journal`) ; `core-rust/` : 7 tests réussis ; Django : test de la pile adapté (le nœud lit Patroni et écrit par HAProxy ; plus de `CLUSTER_PEERS`). L'essai de bout en bout du nœud recentré est joué à la fin de 2d, avec l'alerte.

## Étape 2d — Réplication synchrone, alerte et mesures (2026-10-29)

**Ce qui a été fait.**

- **Mode synchrone selon la taille du cluster** (`orchestrator-go/synchrone/`, option `-synchrone-auto`) : strict à partir de 3 serveurs, non strict en dessous, appliqué par l'orchestrateur élu via `PATCH /config` de Patroni (ADR-006, décision 1). Couverture 73 %, test d'intégration sur etcd réel.
- **Alerte « sous le seuil »** : le nœud remonte l'état du cluster (`etat_quorum`) à chaque battement de cœur et **immédiatement** à chaque changement (au lieu d'attendre jusqu'à 60 s). Le portail l'enregistre (`Device.etat_quorum`, migration 0007), en déduit l'état du tenant (pire état parmi ses serveurs en ligne, `Tenant.etat_cluster`) et alerte :

- **espace PME** : bandeau « Vos données ne sont plus protégées » ou « Votre cluster ne peut plus écrire » ; - **console éditeur** (supervision des clusters) : nouveaux motifs `quorum_lost`, `unprotected`, `no_witness`.

- **Mesures** : campagne de chaos rejouée avec `loop_wait` à 1 s ; résultat et décision dans ADR-006.

**Vérifications.**

- Django : 74 tests réussis (dont 3 nouveaux sur l'état du cluster) ; frontend : build et lint sans erreur ; Go : tests unitaires et d'intégration réussis.
- **Essai sur trois machines** (Docker-dans-Docker, pile générée, images publiées, nœud recentré de 2c) : 13 vérifications sur 13 — premier serveur leader et DEK générée, second en standby synchrone et « en attente d'enrôlement », témoin, nœud voyant le cluster sain, mode non strict à deux serveurs, état « sain » au portail ; puis coupure brutale de la machine du primaire → bascule et écriture par HAProxy en 10 s sans perte, **alerte « sous le seuil » au portail en 1 s** ; retour de l'ancienne machine en standby (20 s) et état redevenu « sain ».
- Journaux du nœud pendant l'essai : « sous le seuil » → « sain » → « quorum perdu » (pendant la bascule) → « sous le seuil » (époque 3) → « sain » ; prise de rôle primaire consignée dans `ss-journal` avec l'époque.

**Reste à faire sur le banc AMANE Test** : mêmes vérifications sur machines réelles (annexe), en particulier les délais de bascule sous Docker Desktop.

## Étape 3 — Le nœud, hôte d'exécution (2026-10-29)

**Ce qui a été fait (ADR-007).**

| Domaine | Fichiers | Contenu |
|---|---|---|
| Manifeste | `tenants/manifeste.py` | Schéma, validation (14 cas de refus testés), mise en forme pour la pile (valeurs YAML sûres, `$` échappé pour Compose) |
| Pile générée | `docker-compose.yml.tmpl` | Service `amane-application` conditionnel : image par digest, lanceur en point d'entrée, commande d'origine, variables, volumes ; rien ne change pour un tenant sans application |
| Orchestrateur | `lanceur/` (couverture 100 %), `cmd/orchestrator/lancer.go`, `provision/`, `bootstrap/` | Sous-commande `lancer` (attend les identifiants, rend les modèles, remplace son processus par la commande d'origine) ; copie du binaire statique par l'amorçage ; rôle et base `application` créés sur le primaire ; noms SQL validés et échappés |
| Nœud | `spike/node/src/application.rs`, `web.rs`, `auth/routes.rs`, `cmd/run.rs` | Reverse proxy (en-têtes de connexion filtrés, `X-Forwarded-*`, redirections laissées au navigateur, réponses en flux) ; console du nœud sous `/amane` ; sonde de santé toutes les 30 s ; état remonté au portail |
| Portail | `devices` (migration 0008) | `Device.etat_application` (`ok` / `ko`) |
| Exemple | `ha-lab/application-exemple/` | SaaS d'exemple qui ne connaît que `DATABASE_URL`, image `FROM scratch` (ni shell ni outils) |

Le rattachement d'une application à la licence d'un tenant relève de l'étape 5 : jusque-là, `application_du_tenant` ne renvoie rien.

**Vérifications.**

- Django : 82 tests réussis, dont 8 sur le manifeste et la pile avec application (service, lanceur, aucun mot de passe dans la pile, nœud configuré, témoin sans application, application sans base).
- Go : `lanceur` 100 %, tests de la sous-commande `lancer` et de la copie du binaire ; suite complète sur etcd réel réussie. Un échec isolé du superviseur est survenu une fois sous la forte charge de l'essai en parallèle (test d'intégration sensible au temps), non reproduit sur deux exécutions suivantes.

- Rust : 20 tests du nœud, dont un relais de bout en bout vers une vraie application HTTP (méthode, chemin, requête, corps, cookies, en-têtes de connexion filtrés) et l'application absente (502).
- **Essai sur trois machines — interrompu.** Claude Code a arrêté l'essai faute de mémoire sur le poste ; il n'a pas été relancé, conformément à la consigne. Ce qu'il avait validé sur la première machine : l'application d'exemple **démarre dans son image** `scratch` (le lanceur injecte `DATABASE_URL`, l'orchestrateur a créé le rôle et la base, l'application les joint par HAProxy), et la sonde du nœud la voit passer d'« indisponible » à « disponible ». Restent à jouer, quand la mémoire le permettra : page servie par le proxy des deux machines, compteur partagé, continuité après bascule (script d'essai conservé hors dépôt, à réintégrer à `ha-lab`).

## Étape 4 — Confinement et chiffrement (2026-10-29)

**Ce qui a été fait (ADR-008).**

| Domaine | Fichiers | Contenu |
|---|---|---|
| Confinement | `docker-compose.yml.tmpl` | Réseau `amane-application` interne, créé seulement si le tenant héberge une application ; l'application n'est que sur lui ; nœud et HAProxy y sont attachés |
| Enrôlement | `spike/node/src/enrolement.rs`, `cmd/enroler.rs`, `auth/routes.rs`, `web.rs` | Codes à usage unique (alphabet sans caractères ambigus, saisie tolérante), page d'administration « Enrôlement », route `POST /amane/api/enrolement` authentifiée par le seul code, commande `ss-node enroler` |
| Sauvegardes | `spike/node/src/sauvegarde.rs`, `cmd/sauvegarder.rs`, `cmd/run.rs`, `node/Dockerfile` | Export, chiffrement par la DEK, dépôt et restauration ; planification dans la boucle de supervision (seulement quand l'écriture est autorisée) ; client PostgreSQL 17 dans l'image du nœud (dépôt officiel PGDG, même version que Spilo) |
| Relais | `spike/relay/src/main.rs` | Routes de blobs corrigées et limite de taille (voir plus bas) ; premiers tests du relais |
| Chiffrement du disque | installeurs Linux et Windows, `devices` (migration 0009), `api_register.py` | Détection LUKS (`lsblk`) et BitLocker (`Get-BitLockerVolume`), avertissement en fin d'installation, `Device.chiffrement_disque` (`oui`, `non` ou `inconnu`) |

**Défauts trouvés et corrigés.**

- **Les routes de blobs du relais n'ont jamais fonctionné.** Elles étaient écrites `/api/blobs/{tenant_id}/{key}`, syntaxe d'axum 0.8, alors que le relais utilise axum 0.7, où les paramètres s'écrivent `:nom` : la route était prise au pied de la lettre et tout dépôt répondait 404. Corrigé et couvert par un test de bout en bout (dépôt de 3 Mo, relecture, suppression).
- `ss-node adduser` ignorait `PG_URL` et ne pouvait pas joindre la base dans la pile (il ne lisait que le fichier de configuration).

**Vérifications.**

- Django : 100 tests réussis (+18), dont le réseau interne (application isolée, seuls nœud et HAProxy attachés, aucun réseau applicatif sans application), la détection du chiffrement dans les deux installeurs et son enregistrement par le portail (valeur invalide refusée).

- Rust : workspace `spike` complet réussi ; nœud 28 tests (+8 : codes à usage unique, annulation après échecs, DEK scellée ouverte par le seul destinataire, clés de rotation, lecture des secrets, blob illisible sans la DEK), relais 2 tests (dépôt de bout en bout, refus des évasions de chemin).
- **Essais légers sur Docker réel** (deux ou trois conteneurs, scripts hors dépôt) :

- sauvegarde, 6/6 : export chiffré déposé ; aucune donnée en clair dans le blob du relais (ni les données, ni l'en-tête `PGDMP`) ; table supprimée puis restaurée depuis le relais ; une autre DEK ne peut pas restaurer ; - enrôlement, 7/7 : code émis par la console du serveur A ; B démarre sans DEK ; un faux code est refusé ; B enrôlé avec le vrai code ; pas de second enrôlement ; la DEK de B est scellée pour sa propre clé ; - confinement : depuis un réseau interne, aucune sortie vers Internet, le voisin du réseau reste joignable.

# 7. Points ouverts

| Sujet | Question | Lié à |
|---|---|---|
| Chiffrement au repos | Tranché par ADR-008 (chiffrement du disque vérifié). À rouvrir si une PME exige un chiffrement sous la DEK indépendant de son système | ADR-008 |
| Licence du SaaS externe | Comment l'application vérifie-t-elle sa propre licence sans accès sortant vers son éditeur ? Relais de métadonnées par le nœud ? | ADR-003 |
| Mises à jour de l'application | Signature des images (cosign) en plus du digest ? Qui valide une nouvelle version ? | ADR-002 |
| Authentification utilisateur | Le reverse proxy ajoute-t-il un contrôle d'appareil appairé devant l'interface, ou laisse-t-on uniquement le login de l'application ? | ADR-003 |
| Applications multi-conteneurs | Un SaaS peut nécessiter plusieurs services (web, worker, cache) : le manifeste doit-il décrire un ensemble ? | ADR-002 |
| Bases autres que PostgreSQL | Que faire d'une application qui exige MySQL ou MongoDB ? | ADR-001 |
| Délais de bascule sur le banc | Confirmer sur machines réelles les cibles d'ADR-006 (laboratoire : crash < 5 s, machine ≈ 5 s, partition < 8 s) | ADR-006, banc |
| Validation sur le banc | Installer AMANE Test (2 serveurs + témoin) selon l'annexe ; vérifier pare-feu Windows et Docker Desktop | Étape 2b |
| Essai de bout en bout de l'étape 3 | Rejouer l'essai sur trois machines avec l'application d'exemple (interrompu faute de mémoire) | Étape 3 |
| WebSockets | Le relais du nœud ne relaie pas encore les connexions WebSocket | ADR-007 |
| Enrôlement sur un réseau hostile | L'échange d'enrôlement passe en HTTP sur le réseau local : la DEK y est scellée, mais un attaquant placé au milieu pourrait capter le code et substituer sa clé publique, ou remettre une fausse DEK. Piste : afficher l'empreinte de la clé du nouveau serveur dans la console pour confirmation humaine, ou TLS du mesh | ADR-008 |
| Mesh WireGuard | Reporté au-delà de l'étape 7 ; à valider sur le banc (`NET_ADMIN`, Docker Desktop) | ADR-008 |
| Taille des sauvegardes | L'export est chiffré en mémoire en un seul bloc (limite pratique : quelques centaines de Mo). Au-delà : chiffrement par morceaux en flux | ADR-008 |
| Jeton du relais | La pile générée ne transmet pas `RELAY_AUTH_TOKEN` au nœud : un relais qui l'exige refuserait les sauvegardes. Le jeton doit être propre au tenant, sinon un tenant pourrait écraser les blobs d'un autre | ADR-008, étape 5 |
| Sauvegarde sur un serveur non enrôlé | Un primaire sans DEK ne sauvegarde pas (il le signale dans ses journaux). Alerter le portail ? | ADR-008 |
| Dossier `pme-deploy/` | Décrit l'ancienne pile V2, désormais remplacée par la pile générée : à retirer ou réécrire | Étape 2c ou 7 |
| Vulnérabilités préexistantes | `govulncheck` signale 12 vulnérabilités atteignables antérieures à 2b : bibliothèque standard (toolchain 1.27.0), `golang.org/x/net`, `google.golang.org/grpc` | Commit de mise à jour dédié |

| Sujet | Question | Lié à |
|---|---|---|
| Couverture Go globale | 68,2 % (68,9 % avant 2b), sous le seuil CI de 70 % depuis avant la refonte : `main()` de l'orchestrateur et `wgmesh` ne sont pas testés | CI |
| Filtres de chemins de `chaos-tests.yml` | Le workflow surveille `consensus/**`, `supervisor/**`… à la racine, alors que ces dossiers sont sous `orchestrator-go/` : il ne se déclenche jamais sur une PR | CI |

# Annexe — Mettre à jour ce document

1. Modifier la source `docs/refonte-v3/AMANE-Refonte-V3.md` : statut des étapes (§ 5), entrée dans le journal (§ 6), nouvelle ADR (§ 3) ou point ouvert (§ 7).
2. Incrémenter la version dans l'en-tête et ajouter une ligne à l'historique des versions.
3. Régénérer le PDF : `python docs/refonte-v3/generer_pdf.py` (dépendance : `reportlab`).
4. Commiter la source et le PDF ensemble, par exemple `docs(refonte): étape 1 — nettoyage`.

# Annexe — Validation sur le banc AMANE Test (étape 2b)

Objectif : installer le banc **AMANE Test** (3 postes) en 2 serveurs + 1 témoin avec la pile générée, puis vérifier la bascule réelle.

**Côté éditeur (portail).**

1. Démarrer la pile de développement (`docker compose up -d`) : portail, registre local (port 5000).
2. Construire l'image du nœud (`spike\build-node.ps1`), puis publier la pile : `bash scripts/publier-pile.sh localhost:5000`. Vérifier que `media/node-images/pile.json` liste six images `@sha256:`.
3. Dans la console, **désactiver les anciennes machines** du tenant AMANE Test. Sinon l'amorçage verrait des machines connues mais injoignables et refuserait, à juste titre, de créer un cluster.

**Côté PME (les trois postes).**

1. Depuis l'espace PME du tenant, télécharger l'installeur **serveur** et le lancer en administrateur sur le poste 1. Noter le mot de passe pgAdmin affiché à la fin.
2. Lancer l'installeur **serveur** sur le poste 2 ; il doit annoncer « 1 serveur(s) déjà installé(s) ».
3. L'espace PME doit alors proposer un témoin : télécharger l'installeur **témoin** et le lancer sur le poste 3.

**Vérifications.**

1. Sur un serveur : `docker exec amane-postgresql patronictl list` → un `Leader` et un `Sync Standby` en `streaming`, nommés d'après les postes.
2. `docker exec amane-etcd etcdctl member list` → 3 membres `started`.
3. Console éditeur : 2 serveurs et 1 témoin actifs ; la licence ne compte que les 2 serveurs.
4. Bascule : éteindre brutalement le poste du primaire (alimentation) ; sur l'autre serveur, `patronictl list` doit le montrer `Leader` en quelques secondes ; pgAdmin (port 5050) doit pouvoir écrire.
5. Rallumer le poste : il doit revenir en `Replica` puis `Sync Standby`.

Consigner les résultats et les durées dans le § 6 (étape 2d pour les mesures fines).